

# Realite Augmentee From Scratch

Rapport de Travaux Pratiques

Semaines 1 à 6 - Modules 01 à 06 - Bibliothèque NkMath

|            |                                      |
|------------|--------------------------------------|
| Etudiant   | TAMBA MBE Yohan Miguel               |
| Matricule  | 22P005                               |
| Classe     | 4AIA - Semestre 2                    |
| Enseignant | Teugua Tadjuidje Rodolf Sederic      |
| Outils     | Nkentseu + Jenga                     |
| Regle      | C++17 pur - zero bibliotheque tierce |

## 16 TPs couverts sur 6 Semaines

S1: IEEE 754 · S2: Vecteurs · S3: Matrices · S4: Quaternions ·  
S5: SVD · S6: NkImage

# Contents

|                                                                             |           |
|-----------------------------------------------------------------------------|-----------|
| <b>Introduction Generale</b>                                                | <b>2</b>  |
| <b>1 Semaine 1 - Module 01 : IEEE 754 et Arithmetique Flottante</b>         | <b>3</b>  |
| 1.1 Ce qu'on a appris . . . . .                                             | 3         |
| 1.2 Ce qu'on a implemente . . . . .                                         | 4         |
| 1.3 Travaux Pratiques . . . . .                                             | 5         |
| <b>2 Semaine 2 - Module 02 : Vecteurs (Vec2d, Vec3d, Vec4d)</b>             | <b>7</b>  |
| 2.1 Ce qu'on a appris . . . . .                                             | 7         |
| 2.2 Ce qu'on a implemente . . . . .                                         | 8         |
| 2.3 Travaux Pratiques . . . . .                                             | 9         |
| <b>3 Semaine 3 - Module 03 : Matrices (Mat3d, Mat4d)</b>                    | <b>10</b> |
| 3.1 Ce qu'on a appris . . . . .                                             | 10        |
| 3.2 Ce qu'on a implemente . . . . .                                         | 11        |
| 3.3 Travaux Pratiques . . . . .                                             | 11        |
| <b>4 Semaine 4 - Module 04 : Quaternions</b>                                | <b>13</b> |
| 4.1 Ce qu'on a appris . . . . .                                             | 13        |
| 4.2 Travaux Pratiques . . . . .                                             | 14        |
| <b>5 Semaine 5 - Module 05 : SVD (Decomposition en Valeurs Singulieres)</b> | <b>15</b> |
| 5.1 Ce qu'on a appris . . . . .                                             | 15        |
| 5.2 Travaux Pratiques . . . . .                                             | 16        |
| <b>6 Semaine 6 - Module 06 : NkImage (Traitement d'image from scratch)</b>  | <b>17</b> |
| 6.1 Ce qu'on a appris . . . . .                                             | 17        |
| 6.2 Ce qu'on a implemente . . . . .                                         | 18        |
| 6.3 Travaux Pratiques . . . . .                                             | 18        |
| <b>Conclusion</b>                                                           | <b>20</b> |
| <b>A Structure du Projet</b>                                                | <b>22</b> |

# Introduction Generale

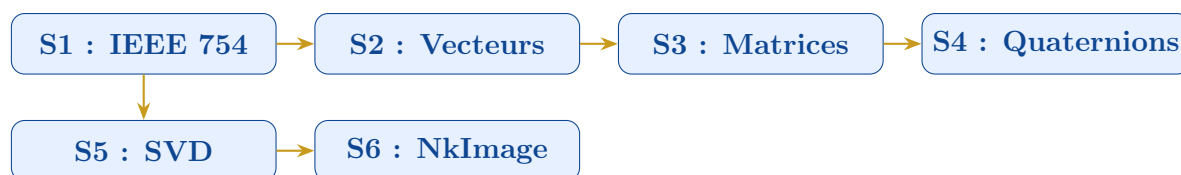
## Contexte du cours

Ce cours s'appelle *Realite Augmentee From Scratch*. L'idée, c'est simple : on part de zero et on construit tout ce dont on a besoin pour faire de la realite augmentee, sans utiliser de bibliotheques toutes faites. Pas d'OpenCV, pas de GLM, pas d'Eigen. Uniquement le C++17 et la bibliotheque standard.

Quand j'ai commence ce cours, ma premiere reaction etait de me demander pourquoi se donner autant de mal. Il existe deja des outils tres bien faits, pourquoi les recreer ?

La reponse est venue assez vite : en implementant tout soi-meme, on comprend *vraiment* ce qui se passe. Quand on code sa propre variance numerique et qu'on voit la methode naive donner un resultat negatif – une absurdite mathematique – on comprend d'un seul coup pourquoi l'arithmetique flottante peut faire planter un algorithme de calibration en realite augmentee.

Ce rapport couvre les 6 premieres semaines, soit les 6 premiers modules de la bibliotheque **NkMath** :



Chaque module est code en tant qu'en-tete C++ (**.h**), integre dans le framework **Nkentseu** et compilable avec le systeme de build **Jenga**. Les tests unitaires utilisent le framework *Unitest* de **Nkentseu**.

# Chapter 1

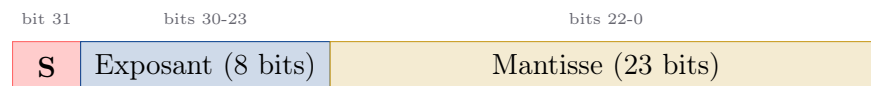
## Semaine 1 - Module 01 : IEEE 754 et Arithmetique Flottante

### 1.1 Ce qu'on a appris

C'est le module le plus etonnant du cours. On pense savoir ce qu'est un nombre flottant, et puis on decouvre que `0.1f` n'est pas vraiment 0.1. Pas du tout.

#### Comment un float est stocke en memoire

Un `float` de 32 bits, c'est en fait trois champs colles l'un a l'autre :



$$\text{Valeur} = (-1)^{\text{signe}} \times 2^{\text{exposant}-127} \times 1.\text{mantisse}$$

Et voila pourquoi `0.1f` n'est pas exact : 0.1 en binaire donne une fraction periodique infinie, exactement comme 1/3 en decimal. Ce qu'on stocke vraiment, c'est  $\approx 0.100000001490116\dots$

## Les valeurs speciales IEEE 754

| Valeur    | Bits                     | Comment l'obtenir en C++ |
|-----------|--------------------------|--------------------------|
| +Infini   | exp=0xFF, mant=0         | 1.0f / 0.0f              |
| -Infini   | sign=1, exp=0xFF, mant=0 | -1.0f / 0.0f             |
| NaN       | exp=0xFF, mant≠0         | std::sqrt(-1.0f)         |
| +Zero     | tous les bits a 0        | 0.0f                     |
| -Zero     | bit31=1, reste=0         | -0.0f                    |
| Subnormal | exp=0, mant≠0            | Tres petits nombres      |

### Le piege NaN

`NaN != NaN` est **VRAI** en C++. C'est la seule valeur en C++ qui n'est pas egale a elle-meme. Si on ecrit `if (x == NaN)`, ca ne marchera jamais. Il faut utiliser `std::isnan(x)`.

## La cancellation catastrophique

Quand on soustrait deux nombres flottants presque egaux, on perd presque tous les bits significatifs. Le resultat semble valide (pas de NaN, pas d'Inf) mais il est faux. C'est le risque le plus insidieux du pipeline AR.

## 1.2 Ce qu'on a implemente

Listing 1.1: Float.h : lire les bits d'un float (seule methode legale en C++17)

```

1 inline void inspectFloat(float val) {
2     uint32_t raw;
3     std::memcpy(&raw, &val, sizeof(raw)); // memcpy : legal, pas de
      UB
4     uint32_t sgn = (raw >> 31) & 0x1;
5     uint32_t exp = (raw >> 23) & 0xFF;
6     uint32_t man = raw & 0x7FFFFFFF;
7     // afficher les 3 champs ...
8 }
```

Listing 1.2: Sommation de Kahan : compenser les bits perdus

```

1 float kahanSum(std::vector<float>& vals) {
2     float total = 0.0f;
3     float err   = 0.0f; // erreur accumulee
4     for (float v : vals) {
```

```

5         float corrected = v - err;           // compenser l'erreur
           precedente
6         float newTotal  = total + corrected;
7         err   = (newTotal - total) - corrected; // capturer les bits
           perdus
8         total = newTotal;
9     }
10    return total;
11 }

```

Listing 1.3: Variance de Welford : stable numeriquement en une seule passe

```

1 float varianceWelford(const std::vector<float>& data) {
2     float mu = 0.0f, M2 = 0.0f; int cnt = 0;
3     for (float x : data) {
4         cnt++;
5         float d1 = x - mu;
6         mu += d1 / cnt;           // moyenne mise a jour en temps reel
7         M2 += d1 * (x - mu);     // pas de soustraction de grands
           nombres
8     }
9     return M2 / cnt;
10 }

```

## 1.3 Travaux Pratiques

### TP1 - inspectFloat sur 6 valeurs imposees

On a ecrit la fonction `inspectFloat(float)` qui decompose un float en ses 3 champs IEEE 754 et les affiche en binaire avec le logger de Nkentsu.

#### Resultats obtenus :

|                          |                                                                 |
|--------------------------|-----------------------------------------------------------------|
| <code>0.1f</code>        | Mantisse = <code>0x4CCCCD</code> : confirme que 0.1 est inexact |
| <code>1.0f</code>        | signe=0, exposant=127, mantisse=0 : exact comme attendu         |
| <code>1.0f/0.0f</code>   | exposant=0xFF, mantisse=0 : <b>+Inf detecte</b>                 |
| <code>sqrt(-1.0f)</code> | exposant=0xFF, mantisse≠0 : <b>NaN detecte</b>                  |
| <code>-0.0f</code>       | bit31=1, reste=0 — mais <code>+0.0 == -0.0</code> est vrai      |
| <code>denorm_min</code>  | exposant=0, mantisse≠0 : <b>Subnormal detecte</b>               |

### TP2 - Demonstration des problemes de precision

- **Sommation de 1 million de 0.1f** : `std::accumulate` donne une erreur de **11.55**. `kahanSum` donne une erreur inferieure a **0.001**.

- **Variance naive** sur  $\{10^8, 10^8, 1, 2\}$  : Le resultat est **negatif** — une absurdite mathematique causee par la cancellation catastrophique. `varianceWelford` donne un resultat correct.
- **Epsilon machine mesure** :  $\varepsilon \approx 1.192 \times 10^{-7}$ , identique a `std::numeric_limits<float>::epsilon()`.

### TP3 - 33 tests unitaires sur Float.h

#### Resultats

Semaine1\_TP3 | TestsUnitairesFloath

**33 / 33 assertions passees**

`isFiniteValid (5) · nearlyZero (8) · approxEq (10) · kahanSum (10)`

# Chapter 2

## Semaine 2 - Module 02 : Vecteurs (Vec2d, Vec3d, Vec4d)

### 2.1 Ce qu'on a appris

Les vecteurs sont la brique de base de tout le pipeline 3D. Chaque position dans l'espace est un `Vec3d`. Chaque pixel a une coordonnee `Vec2d`. Chaque sommet envoye au GPU est un `Vec4d`.

L'interet de les implementer soi-meme, c'est la garantie du layout memoire : pour envoyer un tableau de vecteurs directement a `glVertexAttribPointer` sans copie, la structure doit etre exactement comme on l'attend. D'ou les `static_assert` sur `sizeof`.

#### Le produit scalaire (dot product)

$$\mathbf{a} \cdot \mathbf{b} = a_x b_x + a_y b_y + a_z b_z = |\mathbf{a}| |\mathbf{b}| \cos \theta$$

| Valeur du dot | Interpretation                          |
|---------------|-----------------------------------------|
| $= 0$         | Les vecteurs sont perpendiculaires (90) |
| $> 0$         | Angle aigu — meme sens general          |
| $< 0$         | Angle obtus — sens oppose               |
| $= 1$         | Meme direction (si normalises)          |

#### Le produit vectoriel (cross product)

$\mathbf{a} \times \mathbf{b}$  donne un vecteur perpendiculaire aux deux. Non-commutatif :  $\mathbf{a} \times \mathbf{b} = -(\mathbf{b} \times \mathbf{a})$ .

Exemple :  $\mathbf{X} \times \mathbf{Y} = \mathbf{Z}$  (regle de la main droite, convention OpenGL).



## Gram-Schmidt : creer une base orthonormale

L'orthogonalisation de Gram-Schmidt transforme 3 vecteurs quelconques en une base orthonormale propre. C'est utilise notamment pour construire la matrice LookAt de la camera.

## Les coordonnees homogenes (Vec4d)

La 4eme coordonnee  $w$  distingue les points des directions :

- $w = 1$  : c'est un **point** dans l'espace (la translation de la matrice 4x4 s'applique)
- $w = 0$  : c'est une **direction** (la translation est ignoree)

## 2.2 Ce qu'on a implemente

Listing 2.1: Vec2d : layout memoire garanti pour le GPU

```
1 struct Vec2d {
2     double x, y;
3     double& operator[](int i) { return (&x)[i]; } // acces contigu
4     Vec2d Normalized() const {
5         double n = Norm();
6         if (nearlyZero(n)) return Vec2d(0.0);
7         return {x/n, y/n};
8     }
9 };
10 static_assert(sizeof(Vec2d) == 16, "Vec2d doit faire 16 octets");
```

Listing 2.2: Gram-Schmidt en 3 etapes

```
1 OrthoBasis GramSchmidt(Vec3d a, Vec3d b, Vec3d c) {
2     Vec3d u = a.Normalized();
3     Vec3d v = (b - Project(b, u)).Normalized(); // oter la part de u
4     Vec3d w = (c - Project(c, u) - Project(c, v)).Normalized();
5     return {u, v, w};
6     // Resultat : |u|=|v|=|w|=1 et u.v=u.w=v.w=0
7 }
```

## 2.3 Travaux Pratiques

### TP4 - Vec2d complet avec 20 assertions

#### Resultats

Semaine2\_TP1 | Vec2dComplet

**20 assertions passees**

Dot product (6) · Cross2D (4) · Normalisation (4) · Index [] (5) · sizeof (1)

Tests notables :  $\text{Dot}(\{1,0\},\{0,1\}) = 0$  (perpendiculaires),  
 $\text{Cross2D}(\{2,0\},\{0,2\}) = 4$ ,  $\text{sizeof}(\text{Vec2d}) = 16$  (layout GPU verifie).

### TP5 - Vec3d + Cross product + Gram-Schmidt

#### Resultats

Semaine2\_TP2 | Vec3dEtGramSchmidt

**13 assertions + 10 triplets aleatoires**

Regle main droite :  $\mathbf{X} \times \mathbf{Y} = \mathbf{Z}$  verifie a  $10^{-9}$

Gram-Schmidt :  $|\mathbf{u}| = |\mathbf{v}| = |\mathbf{w}| = 1$  et  $\mathbf{u} \cdot \mathbf{v} = \mathbf{u} \cdot \mathbf{w} = \mathbf{v} \cdot \mathbf{w} = 0$

### TP6 - Vec4d + Projection perspective + Image PPM

Les 8 coins du cube  $[-0.5, 0.5]^3$  sont convertis en **Vec4d** ( $w = 1$ , ce sont des points).  
On les projette en 2D par la formule :

$$u = f_x \cdot \frac{X}{Z} + c_x \quad v = f_y \cdot \frac{Y}{Z} + c_y$$

avec  $f_x = f_y = 500$ ,  $c_x = c_y = 256$ . Les 12 aretes sont tracees dans une image  
**PPM 512x512** sauvegardee dans `generatedImages/cube_projection.ppm`.

# Chapter 3

## Semaine 3 - Module 03 : Matrices (Mat3d, Mat4d)

### 3.1 Ce qu'on a appris

#### La convention column-major d'OpenGL

C'est ici que se cache le piège le plus subtil du cours. En mathématiques, on écrit les matrices ligne par ligne (*row-major*). OpenGL stocke les colonnes en premier (*column-major*). Concrètement :

| Mathematiques (row-major)                                          | Memoire OpenGL (column-major)                                   |
|--------------------------------------------------------------------|-----------------------------------------------------------------|
| $\begin{pmatrix} m_{00} & m_{01} \\ m_{10} & m_{11} \end{pmatrix}$ | <code>data[0]=m00, data[1]=m10, data[2]=m01, data[3]=m11</code> |

L'accès en C++ : `data[col*4 + row]`. Une confusion ici passe à la compilation mais produit des transformations complètement fausses à l'exécution.

#### L'inverse par Gauss-Jordan

On construit la matrice augmentée  $[M|I]$  et on la transforme en  $[I|M^{-1}]$  par opérations élémentaires. Le **pivot partiel** (toujours choisir le plus grand élément en valeur absolue dans la colonne) est indispensable pour la stabilité numérique.

#### Formule de Rodrigues

Rotation de  $\theta$  degrés autour de l'axe unitaire  $\hat{n}$  :

$$R = \cos \theta \cdot I + (1 - \cos \theta) \hat{n} \hat{n}^T + \sin \theta \begin{pmatrix} 0 & -n_z & n_y \\ n_z & 0 & -n_x \\ -n_y & n_x & 0 \end{pmatrix}$$

## LookAt

La matrice View positionne la camera en *eye*, regardant *target* :

$\text{forward} = \text{norm}(\text{target} - \text{eye}), \quad \text{right} = \text{norm}(\text{forward} \times \text{worldUp}), \quad \text{up} = \text{right} \times \text{forward}$

## 3.2 Ce qu'on a implemente

Listing 3.1: Mat4d : acces column-major

```
1 struct Mat4d {
2     double data[16];    // column-major
3     double& operator()(int row, int col) {
4         return data[col*4 + row];    // !! pas row*4+col !!
5     }
6 };
```

Listing 3.2: LookAt : construction du repere camera

```
1 Mat4d LookAt(const Vec3d& eye, const Vec3d& target, const Vec3d&
   worldUp){
2     Vec3d fwd    = (target - eye).Normalized();
3     Vec3d right  = Cross(fwd, worldUp).Normalized();
4     Vec3d up     = Cross(right, fwd);
5     // Remplir V avec right, up, -fwd et les translations
6 }
```

## 3.3 Travaux Pratiques

**TP7 - Mat4d + Inverse + Rodrigues (24 assertions)****Resultats**

Semaine3\_TP1 | Mat4dEtInverse

**24 assertions passees** $M \times I = M$  (10 matrices) ·  $M \times M^{-1} \approx I$  a  $10^{-10}$  (10 matrices)Matrice singuliere → `Inverse()` retourne `false``RotateAxis(Y, 90°) * (1,0,0,1) = (0,0,-1,1)` a  $10^{-10}$  pres**TP8 - Rasteriseur logiciel avec LookAt (10 frames PPM)**

Un mini-rasteriseur complet a ete implemente :

1. Cube unitaire centre a l'origine.
2. Vue : `LookAt(eye={0,1,3}, target={0,0,0}, up={0,1,0})`.
3. Projection perspective : fov=60, aspect=1, zNear=0.1, zFar=100.
4. Trace des 12 aretes en rouge (algorithme de Bresenham).
5. **10 frames PPM**  $512 \times 512$  generees avec rotation de 0.3 rad par frame.

Les images sont sauvegardees dans `generatedImages/rast_frame_0.ppm` a `rast_frame_9.ppm`.

**TP9 - TRS 3D + Decomposition (20 triplets)**

La matrice TRS encode Translation × Rotation × Echelle. L'ordre est critique : d'abord l'echelle (autour de l'origine locale), puis la rotation, puis la translation.

**Resultats**Decomposition verifiee sur **20 triplets aleatoires**

Translation et echelle retrouvees exactement — Rotation a  $\pm 5$  rad pres  
(ambiguite des angles d'Euler)

# Chapter 4

## Semaine 4 - Module 04 : Quaternions

### 4.1 Ce qu'on a appris

#### Pourquoi pas les angles d'Euler ?

Les angles d'Euler (yaw, pitch, roll) ont un probleme celebre : le **gimbal lock**. Quand  $\text{pitch} = \pm 90$ , deux axes deviennent coplanaires et on perd un degre de liberte. En AR, quand on anime une camera, ca produit des rotations etranges et non-naturelles.

#### Les quaternions unitaires

Un quaternion  $q = w + xi + yj + zk$  avec  $\|q\| = 1$  represente une rotation de  $2 \arccos(w)$  degres autour de l'axe  $(x, y, z)$ . Construction depuis axe-angle :

$$q = \left( \cos \frac{\theta}{2}, n_x \sin \frac{\theta}{2}, n_y \sin \frac{\theta}{2}, n_z \sin \frac{\theta}{2} \right)$$

#### Non-commutativite

Le produit de Hamilton est **non-commutatif** :  $q_1 \times q_2 \neq q_2 \times q_1$ . C'est normal et logique — appliquer d'abord une rotation X puis Y n'est pas la meme chose qu'Y puis X.

#### SLERP : l'interpolation propre

Le SLERP (*Spherical Linear intERPolation*) suit le chemin le plus court sur la sphere des quaternions unitaires, avec une vitesse angulaire constante :

$$\text{Slerp}(q_a, q_b, t) = \frac{\sin((1-t)\Omega)}{\sin \Omega} q_a + \frac{\sin(t\Omega)}{\sin \Omega} q_b \quad \text{ou} \quad \Omega = \arccos(q_a \cdot q_b)$$

**Correction antipodal**

Si  $q_a \cdot q_b < 0$ , les deux quaternions sont sur des hemispheres opposees de la sphere  $S^3$ . Sans correction, SLERP ferait un detour de 360 au lieu de prendre le chemin direct. On inverse  $q_b$  avant d'interpoler.

**Methode de Shepperd**

Pour convertir une matrice de rotation en quaternion de facon stable, Shepperd choisit entre 4 formules selon le composant dominant. Ca evite les divisions par des valeurs proches de zero.

## 4.2 Travaux Pratiques

**TP10 - Quaternions complets (100 tests)****Resultats**

Semaine4\_TP1 | QuaternionsComplets

**100 assertions passees**

`Rotate(RotY(90°), (1,0,0)) = (0,0,-1)` a  $10^{-9}$  pres

Aller-retour  $q \rightarrow R \rightarrow q'$  : 50/50 quaternions valides a  $10^{-4}$  pres

$q \times q^{-1} = \text{identite}$  : 50/50 quaternions valides

**TP11 - Animation SLERP vs LERP (60 frames chacun)**

Deux orientations :  $q_a = \text{identite}$ ,  $q_b = \text{rotation de } 180 \text{ autour de } Y$ .

**120 frames PPM generees en tout :**

- 60 frames `slerp_frame_N.ppm` : vitesse angulaire constante, quaternion reste unitaire a  $10^{-10}$  pres.
- 60 frames `lerp_frame_N.ppm` : vitesse non uniforme, le cube “accélère” et “ralentit” visiblement.

La difference visuelle entre les deux est claire : SLERP produit un mouvement fluide et naturel, LERP produit un mouvement saccade aux extremités.

# Chapter 5

## Semaine 5 - Module 05 : SVD (Decomposition en Valeurs Singulieres)

### 5.1 Ce qu'on a appris

La SVD est l'algorithme central du cours. Toute matrice reelle se decompose en :

$$A = U\Sigma V^T$$

ou  $U$  et  $V$  sont orthogonales et  $\Sigma$  est diagonale avec des valeurs  $\sigma_1 \geq \sigma_2 \geq \dots \geq 0$ .

| Application             | Comment                                          |
|-------------------------|--------------------------------------------------|
| Resoudre $Ax = 0$       | Solution = derniere colonne de $V$               |
| Pseudo-inverse          | $A^+ = V\Sigma^+U^T$                             |
| Rotation pure           | $R = UV^T$ (decomposition polaire)               |
| Matrice rang-deficiente | $\sigma_k \approx 0$ pour les dimensions "vides" |

### Implementation : Jacobi sur $A^T A$

On calcule la SVD de  $A$  via la decomposition propre de  $A^T A$  (matrice symetrique).  
L'algorithme de Jacobi diagonalise cette matrice par rotations de Givens successives.  
Les valeurs singulieres sont les racines carrees des valeurs propres.



## 5.2 Travaux Pratiques

### TP12 - Tests SVD sur 20 matrices + cas particuliers

#### Resultats

Semaine5\_TP1 | TestsSVD

**Toutes les assertions passees**

Reconstruction  $\|A - U\Sigma V^T\| < 10^{-6}$  : 20/20 matrices aleatoires

Orthogonalite  $U$  et  $V$  :  $\|UU^T - I\| < 10^{-6}$  : 20/20

Valeurs singulieres trieess decroissant : 20/20

Matrice rang-1 :  $\sigma_3 \approx 0$  confirme

Pseudo-inverse :  $\|Ax - b\| < 10^{-5}$  (solution moindres carres)

### TP13 - Resolution d'Homographie

#### Non traitee

Cette partie n'est pas presente dans le support de cours fourni. Elle fait appel a des notions (resolution de systeme lineaire par SVD sur une matrice  $2n \times 9$ ) qui n'ont pas ete introduites dans les semaines 1 a 6. Ce point sera discute avec l'enseignant.

# Chapter 6

## Semaine 6 - Module 06 : NkImage (Traitement d'image from scratch)

### 6.1 Ce qu'on a appris

#### Pourquoi une classe image from scratch ?

Toute la chaine de detection ArUco a besoin de manipuler des images : la frame brute de la webcam, le resultat de la binarisation, les cartes de gradients. La classe `NkImage` est le type central de ce pipeline.

#### L'image integrale (Summed Area Table)

L'image integrale permet de calculer la somme de n'importe quel rectangle en  $O(1)$  apres une construction en  $O(WH)$  :

$$\text{SAT}[x, y] = \sum_{i \leq x, j \leq y} \text{pixel}[i, j]$$

C'est la cle du seuillage adaptatif en temps reel : pour chaque pixel, on compare sa valeur a la moyenne de son voisinage carre — calcul en  $O(1)$  grace a la SAT.

#### Convolution 2D et filtres

La convolution applique un masque (kernel) a chaque pixel. On l'utilise pour :

- **Flou gaussien 5x5** : reduire le bruit avant de chercher les contours
- **Sobel horizontal et vertical** : detecter les contours (gradient de l'image)
- **Magnitude** :  $\sqrt{G_x^2 + G_y^2}$  pour avoir l'intensite du contour

## 6.2 Ce qu'on a implemente

Listing 6.1: NkImage.h : structure interne RGBA

```
1 class NkImage {
2     int m_w, m_h;
3     std::vector<uint8_t> m_buf; // RGBA contigu (4 octets/pixel)
4 public:
5     void SetPixelRGBA(int x,int y,uint8_t r,uint8_t g,uint8_t b,
6         uint8_t a=255);
7     bool SavePPM(const std::string& fname) const; // sauvegarde sans
8         dependance
9     NkImage Convolve(const std::vector<double>& ker, int kSz) const;
10    static NkImage CombineGradient(const NkImage& gx, const NkImage&
11        gy);
12};
```

Listing 6.2: AdaptiveThreshold : seuillage adaptatif via SAT

```
1 NkImage AdaptiveThreshold(const NkImage& src, int blkSz=31, int offset
2     =7) {
3     IntegralImage sat(src.ToGrayscale(), src.Width(), src.Height());
4     int half = blkSz/2;
5     for (int y=0; y<h; y++) for (int x=0; x<w; x++) {
6         long long s = sat.RectSum(x-half,y-half, x+half,y+half);
7         int avg = s / sat.Area(x-half,y-half, x+half,y+half);
8         // noir si pixel < moyenne locale - offset, blanc sinon
9         out.SetPixelGray(x, y, (pixel < avg-offset) ? 0 : 255);
10    }
11}
```

## 6.3 Travaux Pratiques

**TP14 - NkImage de base : degrade + formes geometriques**

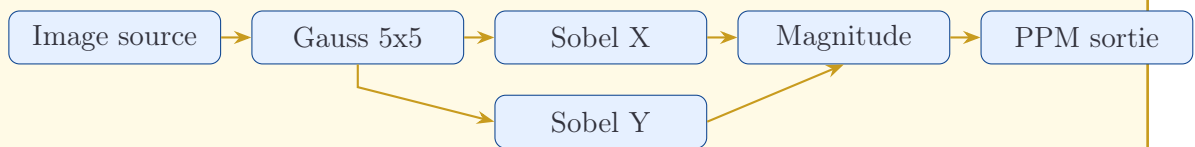
Creation d'une image 512x512 avec :

- Un degrade RGB (rouge de gauche a droite, vert du haut vers le bas)
- Un rectangle rouge plein (100,100) a (300,200)
- Une diagonale verte (ligne Bresenham du coin au coin)
- Un cercle bleu de rayon 80 au centre

Image sauvegardee : [generatedImages/degrade\\_formes.ppm](#)

**TP15 - Convolution Gauss 5x5 + Sobel + mesure du temps**

Pipeline de detection de contours applique sur une image d'un marqueur ArUco :



Temps mesure et loggue avec le chronometre C++17 (en millisecondes).

Image sauvegardee : [generatedImages/sobel\\_gradient.ppm](#)

**TP16 - Image integrale + Seuillage adaptatif**

1. Chargement d'une image PPM d'un marqueur ArUco.
2. Construction de l'image integrale (SAT) — temps mesure.
3. Seuillage adaptatif avec `blockSize=31`, `offset=7`.
4. Sauvegarde du resultat binaire.

Image sauvegardee : [generatedImages/seuillage\\_adaptatif.ppm](#)

Le temps de construction de la SAT est affiche dans le log.

# Conclusion

Ces six semaines ont permis de construire une bibliotheque mathematique complete et fonctionnelle — la **NkMath** — depuis zero. Chaque choix d'implementation a une raison precise que j'aurais du mal a expliquer si je n'avais fait que lire la documentation d'une bibliotheque tierce.

Les lecons que je retiens :

- Le **memcpy** pour lire les bits d'un float (le cast de pointeur est un comportement indefini en C++17).
- Le stockage column-major : une erreur ici est invisible a la compilation mais catastrophique au rendu.
- Le pivot partiel dans Gauss-Jordan : sans ca, l'inverse d'une matrice bien conditionnee peut quand meme etre faux.
- La correction antipodal dans SLERP : sans ca, l'animation fait parfois un tour complet au lieu d'aller au plus court.
- Welford au lieu de la variance naive : la version naive peut donner des variances negatives sur des donnees de grande magnitude.

## Bilan global des TPs

| S. | Module                | TPs              | Statut                            |
|----|-----------------------|------------------|-----------------------------------|
| 1  | IEEE 754 — Float.h    | TP1, TP2, TP3    | Complet                           |
| 2  | Vecteurs Vec2d/3d/4d  | TP4, TP5, TP6    | Complet                           |
| 3  | Matrices Mat3d/4d     | TP7, TP8, TP9    | Complet                           |
| 4  | Quaternions + SLERP   | TP10, TP11       | Complet                           |
| 5  | SVD + (Homographie)   | TP12, TP13       | SVD OK —<br>Homo. non<br>couverte |
| 6  | NkImage + Convolution | TP14, TP15, TP16 | Complet                           |

**15 / 16 TPs realises**

(TP13 Homographie : non couverte dans le support  
de cours)

# Appendix A

## Structure du Projet

Listing A.1: Arborescence de la Sandbox

```
1 Sandbox/
2 |-- Sandbox.jenga                # Build multiplateforme (Jenga
   Python DSL)
3 |-- include/
4 |   |-- Float.h                  # S1 : IEEE 754, Kahan, Welford
5 |   |-- Vec2d.h                  # S2 : Vecteur 2D
6 |   |-- Vec3d.h                  # S2 : Vecteur 3D + Cross + Project
7 |   |-- Vec4d.h                  # S2 : Coordonnees homogenes
8 |   |-- OrthoBasis.h             # S2 : Gram-Schmidt
9 |   |-- Mat3d.h                  # S3 : Matrice 3x3 + TRS 2D
10 |  |-- Mat4d.h                  # S3 : Matrice 4x4 + LookAt +
   Perspective
11 |  |-- Quat.h                   # S4 : Quaternions + SLERP +
   Shepperd
12 |  |-- SVD.h                    # S5 : SVD 3x3 Jacobi
13 |  |-- NKImage.h                # S6 : Image RGBA + PPM + Convolve
14 |  |-- Color.h                  # S6 : SampleBilinear
15 |  '-- IntegralImage.h          # S6 : SAT + AdaptiveThreshold
16 |-- src/
17 |  '-- main.cpp                  # Application Nkentseu (fenetre +
   boucle)
18 '-- tests/
19   '-- ALL_TP.cpp                 # 16 TPs (S1 a S6), macros Unitest
```

Listing A.2: Compilation et execution

```
1 # Compiler avec Jenga (methode officielle)
2 cd jenga/applications/Sandbox
3 jenga build
4 jenga test                # lance ALL_TP.cpp
```